Partie 5
La récursivité

INFO410 – Structures de données classiques

1. Principe

- Une procédure récursive est une procédure qui **se rappelle elle-même**
- De même pour les fonctions

- Toute procédure récursive comporte une **instruction** (ou un bloc d'instructions) qui indique que le reste des instructions **ne doit plus** être exécuté
- C'est le « point terminal », le « point d'appui » ou le « **point d'arrêt** »

1. Principe

- Exemple classique avec la fonction factorielle,

```
function facto(N: integer) return integer is
begin
   if N = 0 or N = 1 then
      return 1;
   else
      return N * facto(N - 1);
   end if;
end facto;
```

Point d’arrêt

1. Principe

- La récursivité utilise toujours la **pile** du programme en cours
- On appelle pile, la **zone mémoire réservée** à chaque programme lorsqu'il est exécuté

- Son rôle est de **stocker** et **empiler** les variables locales et les paramètres d'une procédure ou d'une fonction
- Dans une procédure récursive, les variables locales sont **stockées** et **empilées** autant de fois qu'il y a d'appels récursifs
- Si on ne fait pas attention, on peut parfois arriver à un **débordement**

1. Principe

- Retour de l'exemple classique avec la fonction factorielle,

```
procedure principale is
   …
begin
   put(facto(6));
end principale;
```

facto(1)	1
facto(2)	2
facto(3)	6
facto(4)	24
facto(5)	120
facto(6)	720

120 * 6

2. Exemples

- Qu'affiche `count(3)` ?

```
procedure count(N: in integer) is
begin
   if N = 0 then
      put('.');
   else
      put(N); put('(');
      count(N – 1);
      put(')'); put(N);
   end if;
end count;
```

2. Exemples

- Compter les éléments d’une liste de manière récursive

```
function nb_element(L: in liste) return integer is
   …
end nb_element;
```

3. Version terminale

- On dit qu'une procédure p est récursive terminale quand tout appel récursif est de la forme return p(…);
- Autrement dit, la valeur retournée est directement la valeur obtenu par un appel récursif sans opération supplémentaire sur cette valeur
- De même pour les fonctions

```
return N * facto(N - 1);
```

n'est pas terminal

3. Version terminale

- L'idée est d'ajouter un paramètre qui va jouer le rôle d'accumulateur
- Retour du retour de l'exemple classique avec la fonction factorielle,

```
function facto_t(N: in integer; R: in integer) return integer is
begin
   if N = 0 or N = 1 then
      return R;
   else
      return facto_t(N – 1, N * R);
   end if;
end facto_t;
```

3. Version terminale

- Cela implique d'initialiser l'accumulateur lors de l'appel,

```
put(facto_t(6, 1));
```

- Ou d'avoir une fonction auxiliaire qui le fait directement,

```
function facto(N: in integer) return integer is
begin
   return facto_t(N, 1);
end facto;
```

3. Version terminale

- Il est facile de **dérécursiver** une procédure (ou une fonction) récursive terminale, c'est-à-dire de la transformer en sa version itérative

```
function facto_iter(N: in integer) return integer is
   R: integer := 1;
begin
   while N > 1 loop
      R := R * N;
      N := N – 1;
   end loop;
   return R;
end facto_iter;
```

3. Version terminale

- Compter les éléments d’une liste de manière récursive terminale

```
function nb_elm_t(L: in liste; R: in integer) return integer is
   …
end nb_elm_t;
```

4. Application : la tour de Hanoï

- La tour de Hanoï est un jeu qui consiste à déplacer des disques de diamètres différents d'une tour de départ à une tour d'arrivée en passant par une seule tour intermédiaire
- On ne peut déplacer qu'un seul disque à la fois
- Si on place un disque sur un autre, il doit être forcément plus petit

4. Application : la tour de Hanoï

- On peut résoudre ce problème de manière récursive terminale

```
procedure hanoi(N: in integer; D: in tour; A: in tour) is
   I: tour := …;
begin
   if N > 0 then
      hanoi(N – 1, D, I);
      deplacer_disque(D, A);
      hanoi(N – 1, I, A);
   end if;
end hanoi;
```

tour d'Arrivée

tour de Départ

Nbre de disques

tour Intermédiaire